

SOFTWARE ENGINEERING

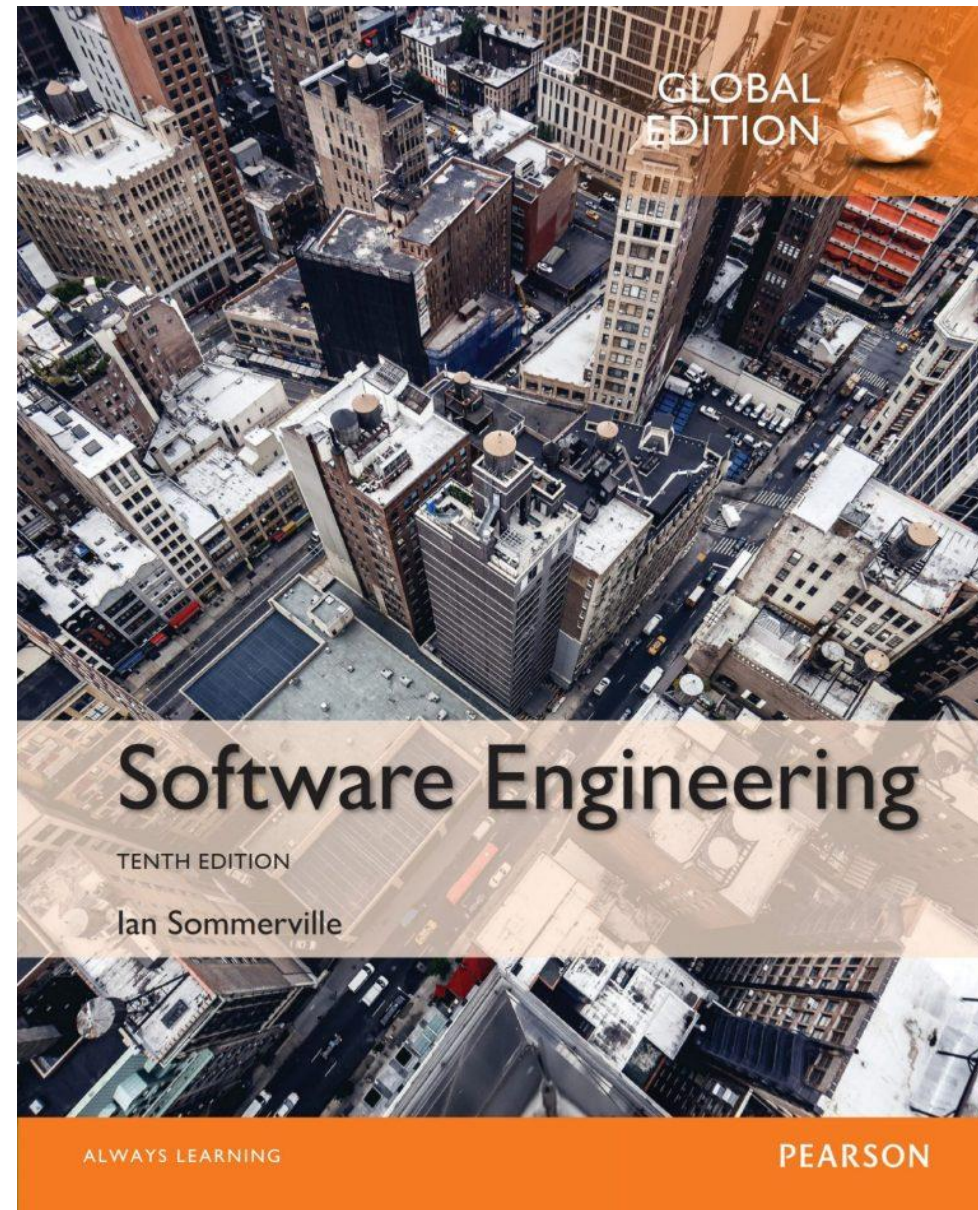
Introduction

By

Dr. Rasha Mahmoud Kamel

Textbook

- Sommerville, Ian. "Software engineering tenth edition.", 2016.



Contents

■ Professional software development:

- Understand what software engineering is and why it is important
- Understand that the development of different types of software system may require different software engineering techniques

■ Software engineering ethics:

- Understand ethical and professional issues that are important for software engineers

■ Case studies:

- Introduce four systems, of different types, which are used as examples throughout the course

What is software?

- A program is an executable code, which serves some computational purpose.
- Many people think that software is simply another word for computer program.
- However, software encompasses more than just the programs themselves.
- A professionally developed software system may include
 - Multiple separate programs.
 - Configuration files used to set up and customize these programs.
 - System documentation that describes the system's architecture and components.
 - User documentation that explains how to use the system.
 - A website for users to download updates, patches, or product information.

What is software?

- If you are writing a program for personal use only, you don't have to worry about writing user guides, documenting the program design, and so on.
- However, if you are writing software that other people will use and other engineers will change, then you typically have to provide additional information beyond the code itself—such as documentation, usage instructions, and design details.

Software Products

■ There are two kinds of software product:

(1) Generic software:

- These are stand-alone systems that are produced by a development organization and sold on the open market to any customer who is able to buy them.
- Examples of this type include apps for mobile devices, software for PCs such as databases, word processors, drawing packages, and project management tools.
- In generic products, the organization that develops the software controls the software specification.
- This means that if they run into development problems, they can rethink what is to be developed.

Software Products

- There are two kinds of software product:

(2) Customized software:

- These are systems that are commissioned by and developed for a particular customer.
- Examples of this type of software include control systems for electronic devices, systems written to support a particular business process, and air traffic control systems.
- For custom products, the specification is developed and controlled by the organization that is buying the software.
- The software developers must work to that specification.

Essential Attributes of Good Software

- **Acceptability:** Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable, and compatible with other systems that they use.
- **Dependability:** Software dependability includes a range of characteristics including reliability, security, and safety. Dependable software should not cause physical or economic damage in the event of system failure.
- **Security:** Software has to be secure so that hackers cannot access or damage the system.
- **Efficiency:** Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, resource utilization, etc.
- **Maintainability:** Software should be written in such a way that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.

Software Engineering

- Software engineering is an engineering discipline that is concerned with the development of software product from the early stages of system specification to maintenance after deployment.
- Software engineers adopt a systematic and well-defined scientific principles, methods and procedures to produce software with the required quality within schedule and budget.
- They apply theories, methods, and tools where appropriate. They always try to discover solutions to problems even when there are no applicable theories and methods.
- Covering all aspects of software production means that software engineering is not just concerned with the technical development processes.
- It also includes activities such as software project management and the development of tools, methods, and theories that support software development.

Why Software Engineering?

- Software engineering is essential for the functioning of government, society, and national and international businesses and institutions.
- We can't run the modern world without software. National infrastructures and utilities are controlled by computer-based systems, and most electrical products include a computer and controlling software.
- Industrial manufacturing and distribution is completely computerized, as is the financial system.
- Entertainment, including the music industry, computer games, and film and television, is software-intensive.
- Almost all of the world's population have a software-controlled mobile phone and almost all of these are Internet-enabled.

Why Software Engineering?

■ The Ariane 5 Disaster (1996)

- On June 4, 1996, the European Space Agency's Ariane 5 rocket exploded just 40 seconds after launch.
- This failure destroyed the rocket and results in a loss about \$370 million.

■ What went wrong? (The software bug)

- The rocket reused software from the earlier Ariane 4 rocket.
- In the software, a 64-bit floating-point number (representing horizontal velocity) was converted into a 16-bit signed integer.
- But Ariane 5 was much faster than Ariane 4 → the value was too large to fit in 16 bits. This caused an overflow error.
- The error triggered the inertial reference system (IRS) to shut down.

Importance of Software Engineering

- The need of software engineering arises because of several factors:
 - **Rapid Change in User Requirements & Environment:** Users' needs and the technologies they use evolve constantly. For example, a messaging app originally designed for text-only must now support voice, video, stickers, and real-time syncing across devices. Software engineering provides a structured way to manage these changes without breaking the existing system.
 - **Large software:** It is easier to build a wall than to build an entire house. Similarly, as the software grows in size and complexity, it becomes difficult to manage without engineering discipline. Software engineering helps by breaking the system into manageable modules, applying industry standards, and uses scientific principles to build complex software reliably.

Importance of Software Engineering

- The need of software engineering arises because of several factors:
 - **Scalability:** If the software process were not based on scientific and engineering concepts, it becomes difficult to scale up (e.g., to add new users, features, or data) and it would be easier to re-create new software from scratch than to scale an existing one.
 - **Dynamic Nature:** Software must adapt to changes in its environment (like new platforms, devices, or regulations). For example, a payment app must adapt to new financial regulations or security standards. Software engineering ensures that the software is flexible and maintainable, allowing for quick updates and improvements.
 - **Cost:** Hardware has become cheaper due to mass production and innovation, but software can still be expensive if not built efficiently. By following systematic processes, software engineering reduces development costs, avoids rework, and delivers software faster and more predictably.

Importance of Software Engineering

- The need of software engineering arises because of several factors:
 - **Quality Management:** Without proper process of software development, the quality of software can't be guaranteed. Software engineering applies quality assurance, testing, code reviews, and documentation to ensure the final product is reliable, secure, and user-friendly.

Software Process Activities

- The systematic approach that is used in software engineering is sometimes called a *software process*.
- A software process is a sequence of activities that leads to the production of a software product. The fundamental activities that are common to all software processes are:
 - (1) **Software specification**, where customers and engineers define the functionality of the software that is to be produced and constraints on its operation.
 - (2) **Software development**, where the software that meets the specification is designed and programmed.
 - (3) **Software validation**, where the software is checked to ensure that it does what the customer wants.
 - (4) **Software evolution**, where the software is modified to meet changing customer needs.

Computer Science vs Software Engineering

■ What is the difference between software engineering and computer science?

- Computer science is concerned with the theories and methods that underlie computers and software systems.
- Software engineering is concerned with the practical problems of developing and producing software.
- Some knowledge of computer science is essential for software engineers in the same way that some knowledge of physics is essential for electrical engineers.

Software Engineering vs System Engineering

■ What is the difference between software engineering and system engineering?

- System engineering is concerned with all aspects of the development and evolution of complex systems including hardware development, software engineering, process design, and system deployment.
- Software engineering is part of this more general process.
- System engineers are involved in specifying the system, defining its overall architecture, and then integrating the different parts to create the finished system.

Software Engineering Diversity

- Software engineering is a systematic approach to the production of software that takes into account
 - The needs of software customers
 - Practical cost
 - Schedule
 - Dependability issues
- The specific methods, tools, and techniques used depend on
 - The type of software
 - The organization developing the software
 - The people involved in the development process

Software Engineering Diversity

- There are no universal software engineering methods that are suitable for all systems and all companies.
- Rather, a diverse set of software engineering methods and tools has evolved over the past 50 years.
- The most significant factor in determining which software engineering methods and techniques are most important is the type of application being developed.

Software Application Types

- There are many different types of application, including:

(1) Stand-alone applications:

- These are application systems that run independently on a personal computer or a mobile device.
- They include all the necessary functionality and typically do not require a network connection to operate.
- Examples of such applications are office applications on a PC (such as word processors and spreadsheets), CAD (Computer-Aided Design) programs, photo editing software, travel planning apps, and so on.

Software Application Types

(2) Entertainment systems:

- Designed for personal use, these systems aim to entertain users.
- Most of these systems are games that run on PCs, mobile devices, or dedicated consoles.
- The most important distinguishing characteristic of entertainment systems is the quality of user interaction and visual/auditory experience.

(3) Embedded control systems:

- These are software systems designed to control and manage hardware devices.
- These systems are among the most common types of software, found in a wide range of everyday devices.
- Examples of embedded systems include the software in mobile phones, software in washing machines, and software in microwave ovens that control the cooking process.

Software Application Types

(4) Interactive transaction-based applications:

- These are applications that run on remote computers and are accessed by users from their own computers, phones, or tablets.
- These include web applications such as e-commerce platforms, where users interact with a remote server to buy goods and services.
- This category also includes business systems, where a business provides access to its systems through web browsers or dedicated client applications and cloud-based services, such as mail and photo sharing.
- Such interactive applications often incorporate a large data store that is accessed and updated in each transaction.

Software Application Types

(5) Batch processing systems:

- These are business systems that are designed to process large volumes of data by processing them in batches rather than individually.
- These systems take in a large number of inputs and create corresponding outputs in a single run.
- Examples of batch systems are periodic billing systems, such as phone billing, and payroll systems for salary payments.

Software Application Types

(6) Systems for modeling and simulation:

- These are systems that are developed by scientists and engineers to model and analyze physical processes or complex situations, which include many separate, interacting objects.
- These systems are often computationally intensive and typically require high-performance parallel computing systems for efficient execution.

Software Application Types

(7) Data collection and analysis systems:

- These are systems that collect data from their environment—often through sensors—and send that data to other systems for processing.
- These softwares are frequently deployed in challenging environments, such as inside engines or in remote locations.
- Large-scale (“big data”) analysis may be performed using cloud-based platforms that apply statistical techniques to identify patterns and relationships in the collected data.

Software Application Types

(8) Systems of systems:

- These are systems, used in enterprises and large organizations, that are composed of a number of other software systems.
- Some of these may be generic software products, such as an Enterprise Resource Planning (ERP) systems, while others are custom-built to meet specific organizational needs.

Software Engineering Fundamentals

- There are software engineering fundamentals that apply to all types of software systems:
 - Software should be developed using a well-managed and clearly understood process. The organization developing the software should plan the development process and have clear ideas of what will be produced and when it will be completed. Naturally, the specific process used depends on the type of software being developed.
 - Dependability and performance are important for all types of systems. Software should behave as expected, without failures, and should be available for use when it is required. It must be safe in its operation and, as much as possible, should be secure against external attacks. Additionally, the system should perform efficiently and should not waste resources.

Software Engineering Fundamentals

- There are software engineering fundamentals that apply to all types of software systems:
 - Understanding and managing the software specification and requirements (what the software should do) is important. It is important to identify what different customers and users expect from the system, and to manage their expectations too ensure that a useful system can be delivered on time and within budget.
 - It is important to make effective use of existing resources. This means that, where appropriate, reuse software components that have already been developed rather than building new ones from scratch.

Software Engineering Challenges

- There are some common challenges that appear across different software types, including:
 - **Heterogeneity:** This refers to the increasing need for systems to operate as distributed systems across networks that include different types of computers and mobile devices. As well as running on general-purpose computers, software may also need to run on mobile phones, tablets, and other platforms. Furthermore, new software often needs to be integrated with older software systems (called legacy systems) written in different programming languages. The key challenge is to develop techniques for building dependable software that is flexible enough to cope with this diversity.
 - **Scale:** Software has to be developed across a very wide range of scales, from very small embedded systems in portable or wearable devices through to Internet-scale, cloud-based systems that serve a global community.

Software Engineering Challenges

- **Business and social change:** Businesses and society are changing incredibly quickly as emerging economies develop and new technologies become available. They need to be able to change their existing software and to rapidly develop new software. Many traditional software engineering techniques are time consuming, and delivery of new systems often takes longer than planned. They need to evolve so that the time required for software to deliver value to its customers is reduced.
- **Security and trust:** As software is intertwined with all aspects of our lives, it is essential that we can trust that software. This is especially true for remote software systems accessed through a web page or web service interface. We have to make sure that malicious users cannot successfully attack our software and that information security is maintained.
- To address these challenges, we will need new tools and techniques as well as innovative ways of combining and using existing software engineering methods.

Software Engineering Ethics

- As a software engineer, you must behave in an ethical and responsible way if you are to be respected as a professional engineer. Many aspects of professional conduct are not governed by law but by a sense of ethical duty. Some of these are:
 - **Confidentiality:** You should respect the confidentiality of your employers or clients, even if no formal agreement has been signed.
 - **Competence:** You should not misrepresent your skills or knowingly accept work that is outside your qualifications or expertise.
 - **Intellectual property rights:** You should be familiar with local laws governing the use of intellectual property such as patents and copyright. You should ensure that the intellectual property of employers and clients is protected.
 - **Computer misuse:** You should not use your technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine) to extremely serious (spreading malware or viruses).

Case Studies

■ The system types that are used as case studies are:

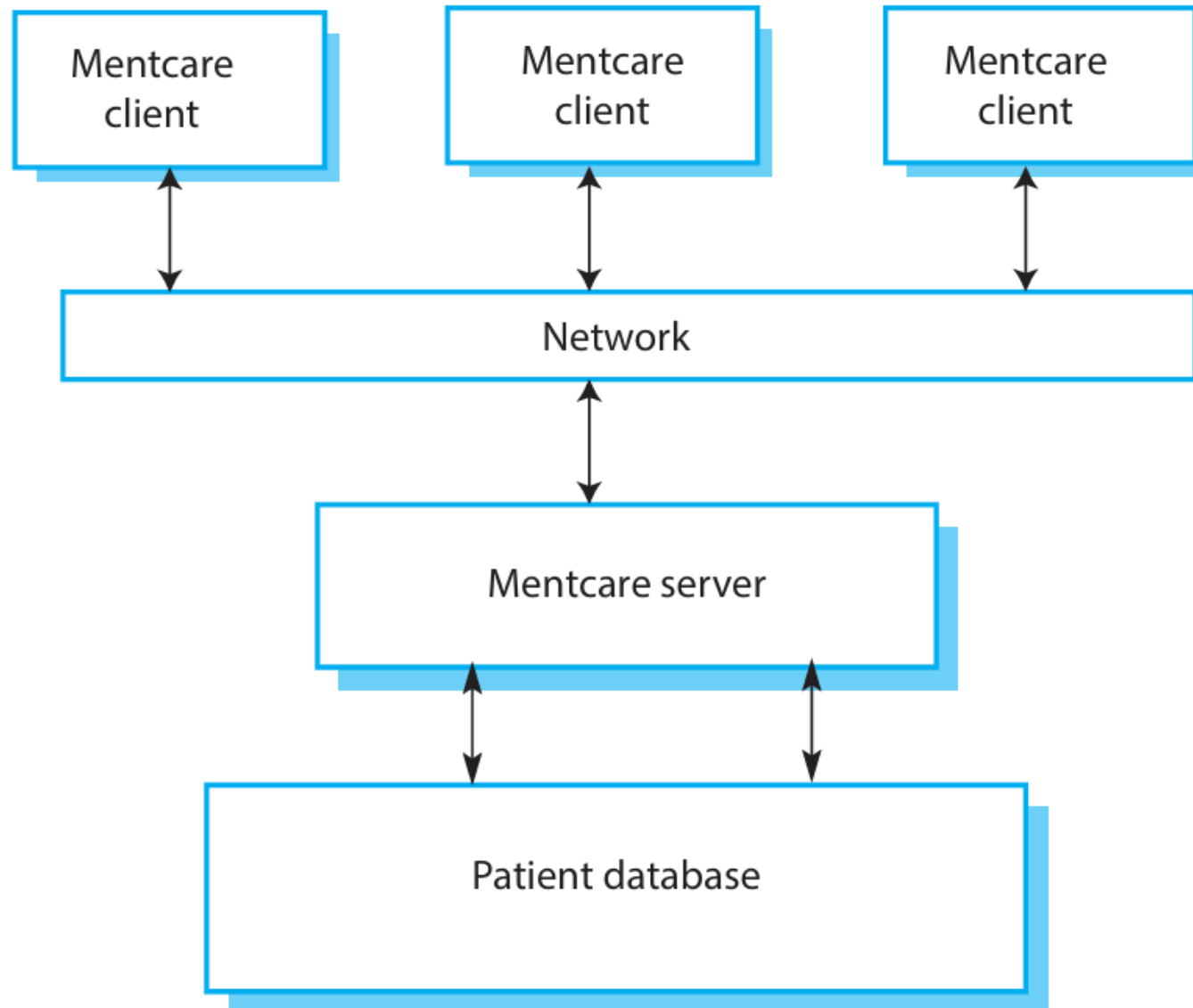
- (1) **An embedded system:** This is a system where the software controls some hardware device and is embedded in that device. Issues in embedded systems typically include physical size, responsiveness, and power management, etc. The example of an embedded system is a software system to control an insulin pump for people who have diabetes.
- (2) **An information system:** The primary purpose of this type of system is to manage and provide access to a database of information. Issues in information systems include security, usability, privacy, and maintaining data integrity. The example of an information system used is a medical records system.

Case Studies

■ The system types that are used as case studies are:

- (3) **A sensor-based data collection system:** This is a system whose primary purposes are to collect data from a set of sensors and to process that data in some way. The key requirements of such systems are reliability, even in hostile environmental conditions, and maintainability. The example of a data collection system is a wilderness weather station.
- (4) **A support environment:** This is an integrated collection of software tools that are used to support some kind of activity. Programming environments, such as Eclipse will be the most familiar type of environment for readers of this book. I describe an example here of a digital learning environment that is used to support students' learning in schools.

Mentcare System



Thank
You